

Dimensionality Reduction

DSC 340 Applied Machine Learning

PDSH 5.07

April 7

THE PLAN

- Quiz
- **What is Dimensionality Reduction?**
- Projection (dropping out features)
- **Principal Component Analysis (PCA)**
- Independent Component Analysis
- Related Methods
- PCA in Python

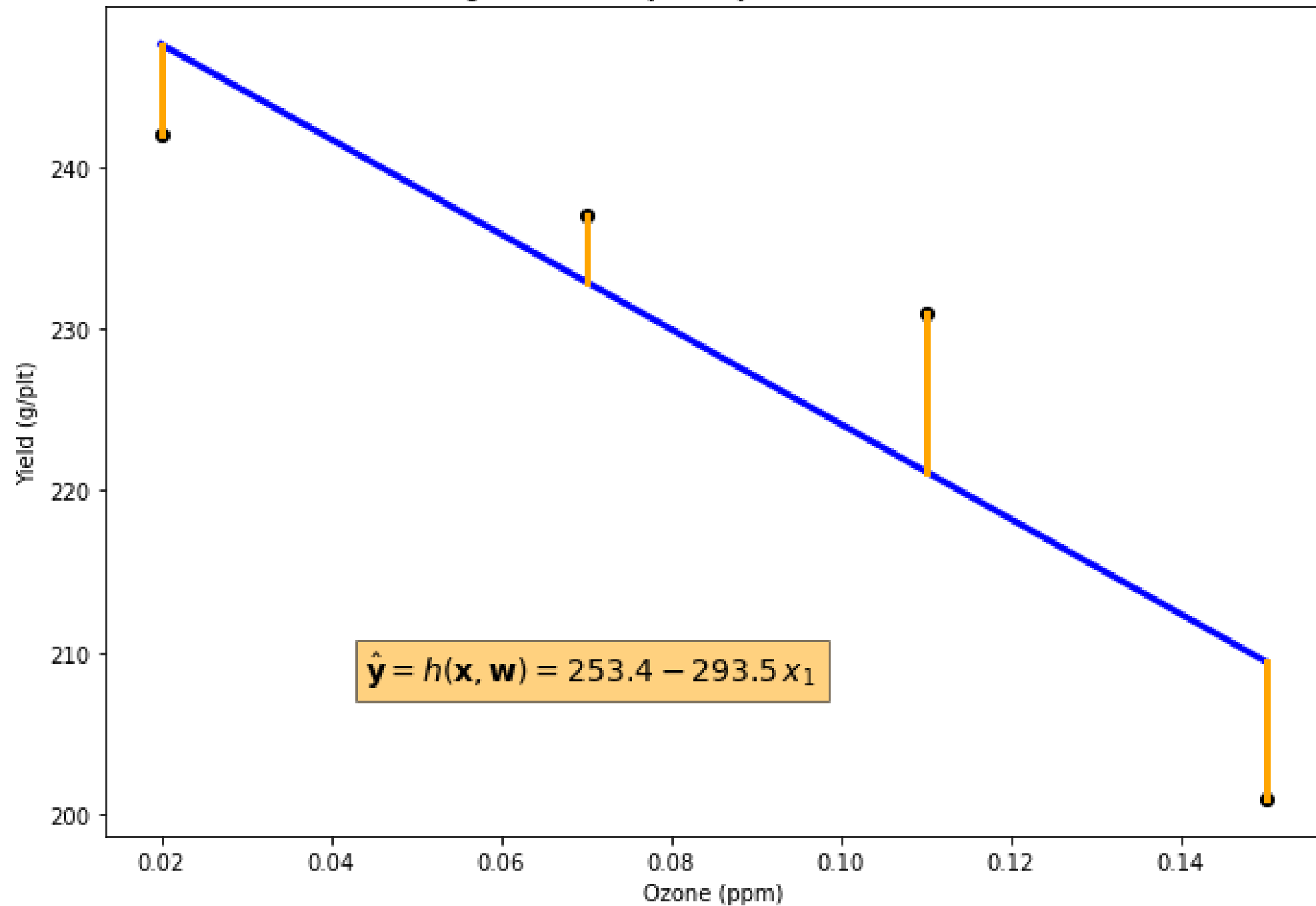
What is dimensionality reduction?

- Recall: conceive our data X as "projected" into **Cartesian space**
- For every **feature**, there is a corresponding **dimension**: D dimensions
 - Works well in 1-3 dimensions
 - Generalizes via linear algebra (MAT 240)
 - In practice, can often just think of as "columns" in dataset
- What if a feature is **irrelevant** to our problem?
- What if a feature is **redundant**? (i.e., highly **correlated** to another feature)
- We could just **omit** those features (**drop columns**): DSC 205
- Mathematically, this is **projecting our data onto a subspace** ...

Generalizing projection

- **Projection onto subspace** = form of **dimensionality reduction**
 - We saw this with **decision forests**
 - Random sample of features: projected onto subspace
 - Informal: Can imagine "flattening" our data from 3D down to 2D or 1D
- Can we generalize this to **oblique projections**?
 - **Oblique** means at an angle, not directly along an axis
 - Instead of looking at a cube head-on, suppose we rotated it, cast shadow
- Consider that this is what **linear** and **logistic regression** do!
 - Projects data **X** onto ONE dimension: vector **w**
 - Logistic then squishes it into $[0,1]$ to give us $P(1|\mathbf{x})$

Regression of soybean yield on ozone level



Transforms in Data Mining

- In data science, we have many techniques: discretization, transforms
- To solve problems, need to look at data from different points of view
- Example: we could **add** two columns, $A + B$, to get new column C
- More generally, new column a linear combination: $C = aA + bB$
- A **projection** is a particular kind of **transform** or **mapping** to our data
- Specifically (in DSC 305) it is a **linear combination** of existing features

$$\mathbf{z} = \mathbf{x} \cdot \mathbf{w} + w_0$$

- Could even think of this as an "axis" in some new coordinate system...

Coordinate Systems

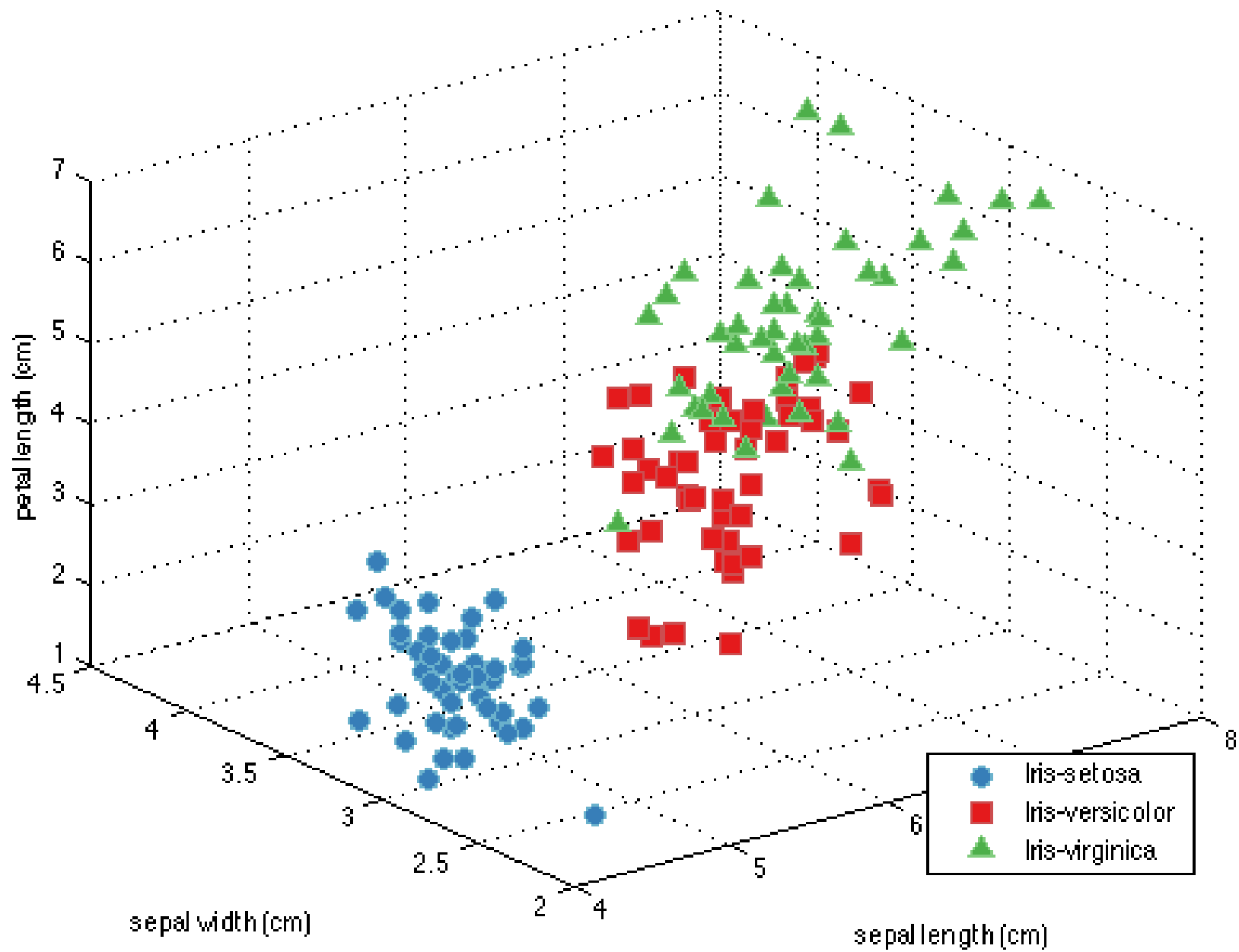
- Suppose we are visualizing data for a presentation (figure)
 - Example: stars in the sky, swarm of flies frozen in time ...
 - 2D scatter plot of X vs. Y ...
 - Map of the world
- **Consider that our choice of axes is completely arbitrary**
 - Maybe a particular choice is easier for our audience to comprehend ...
 - But mathematically, it is all the same
- Could rotate image of stars or flies, switch X and Y, or put south "up"
- In other words, **you could change the coordinate system.**

Is there a best coordinate system?

***There is nothing to stop you ...
from transforming your data into a different coordinate system!***

- Suppose we could choose ANY coordinate system
- Is there is a best choice?
- We could argue that the data itself should provide the answer ...
- Discuss: Photo of a group of people: which perspective is best?
- Can we generalize this to data ... in general?

UCI Iris Data

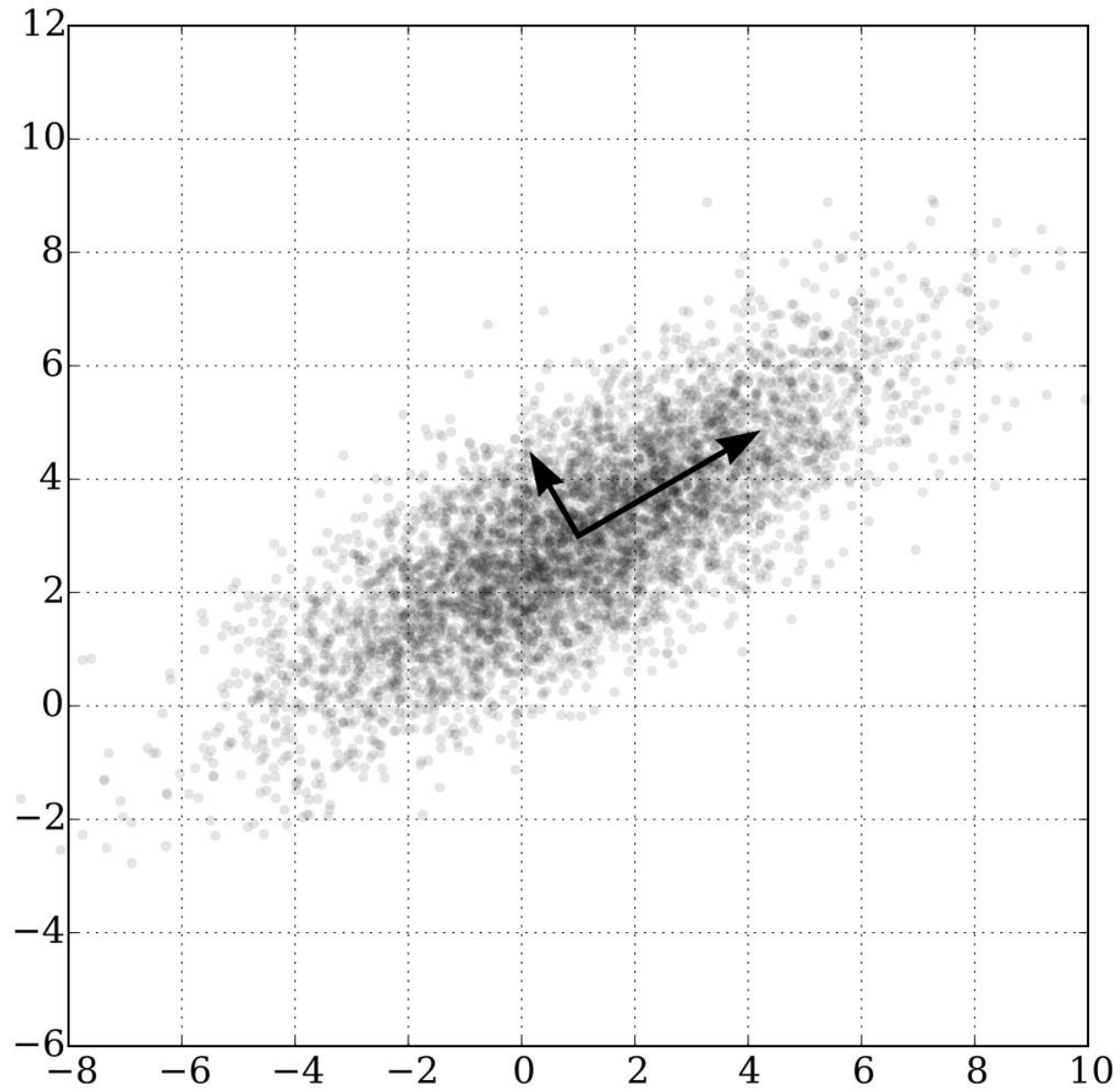


Coordinate systems and variance

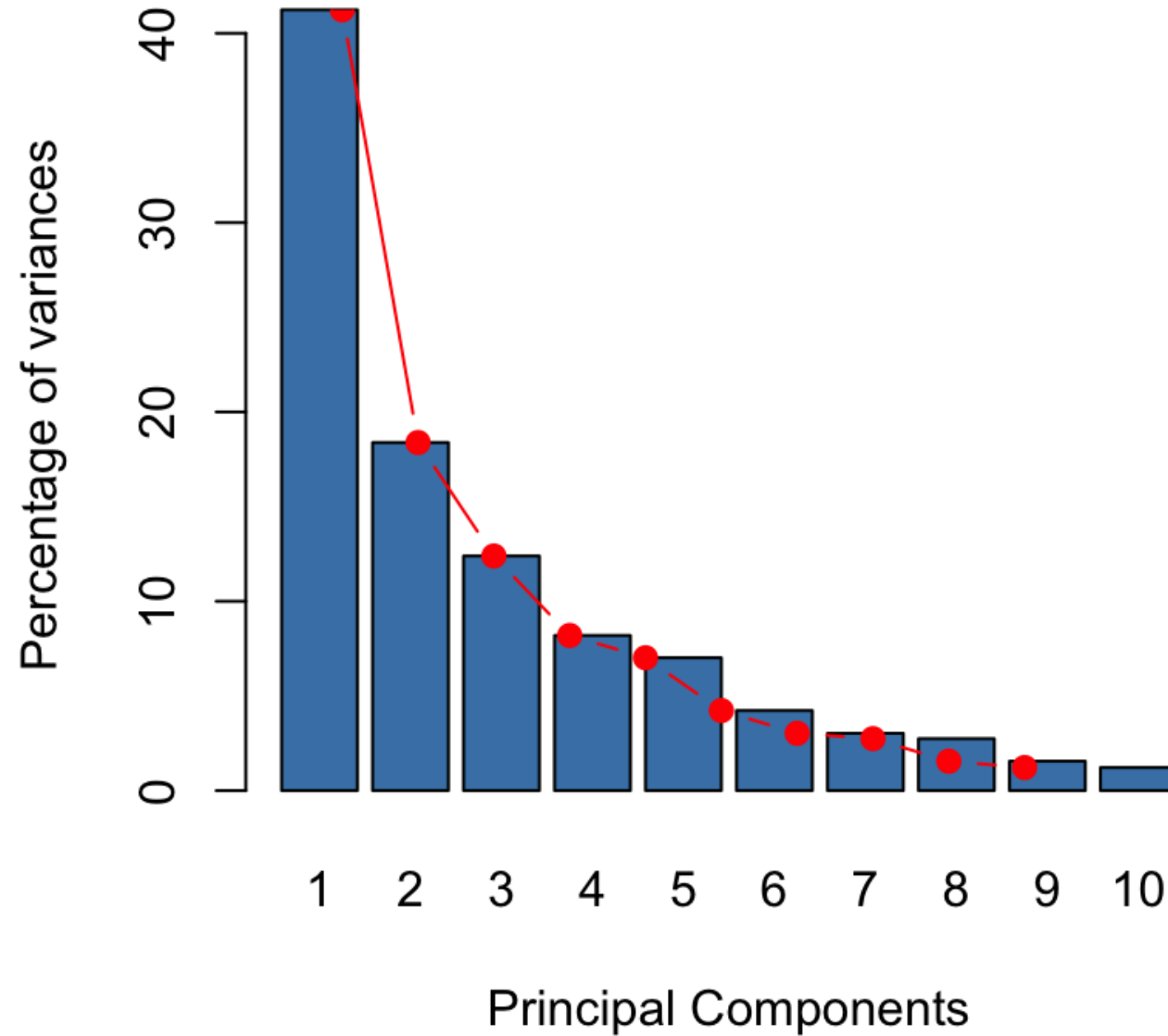
- Consider that whatever coordinate you view data from:
- There is a **variance** to the data
- **Variance is how much the data is "spread out" around the mean**
- Curious fact: add up variance for every coordinate ...
- It turns out that the sum is always the same.
- This is **true provided that** our coordinates are **orthogonal**
- That is, the axes must be at **right angles** to each other everywhere

Principal Component Analysis

- Big idea: objectively, we want the "perfect" coordinate system
- First axis: the one that **maximizes the variance** along that axis
- Second axis: must be **perpendicular** to the first (**orthogonal**)
- Problem: If data has >2 features, we have infinitely many choices!
- How choose? Again, pick the one that **maximizes** remaining **variance**
- Repeat ... in the end there will still be D axes ...
- Probably not our original D features ... but linear combinations ...
- In other words, **each new feature is a projection onto a new axis**



Variances

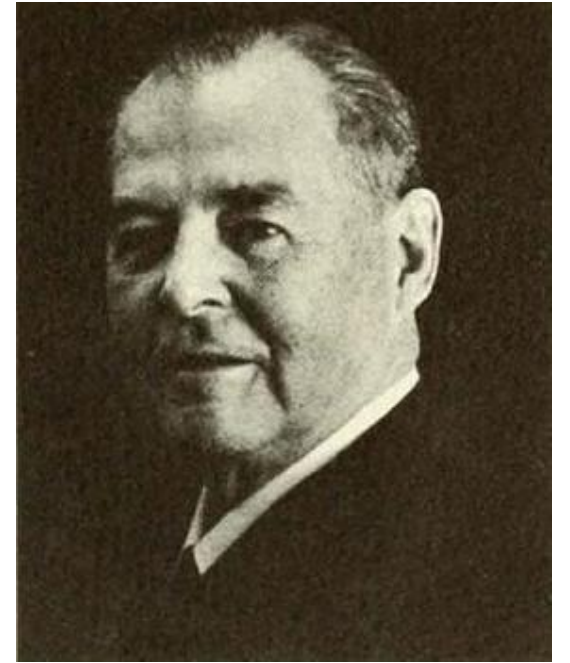


Mathematical Foundation

- In linear algebra, these "new features" are known as **eigenvectors**
- These are **sorted** by their **eigenvalues** ...
- **Eigenvalue** give the **variance** along that new axis
- Obtain through **singular value decomposition (SVD)** of a matrix
- Axes known as **components** because they "account for" the variance
- **Principal** components since we may just care about the first few!
- Usually, we want only the first K components, where $K \ll D$
- Thus, PCA is a common way to go about **dimensionality reduction**

History

- [Harold Hotelling](#) (1895 – 1973)
 - American mathematical statistician and economist
 - Systematized PCA (1933)
 - Defined as: **maximizing variance of projected data**
- [Egon Pearson](#) (1895 – 1980)
 - British statistician: contrib. biometrics, econometrics
 - Defined as: **minimize average projection cost** (1901)
 - Cost = mean sq. dist. of data to their projections
- Turns out that these are equivalent ideas
- Eigenvectors, eigenvalues: Euler, Cauchy, Laplace ...



Assessment

- Advantages:
 - Automatically removes correlated features
 - Can improve the performance of other learning methods
 - Useful for data visualization (e.g., IRIS data from 4D to 2D to show classes)
- Disadvantages:
 - Sensitive to outliers (but robust methods such as RandomizedPCA exist)
 - Less interpretable (original features have meaning: but what are these?)
 - Loss of information (but in data science, often "less is more")
- Caveats:
 - Must scale, discretize, and one hot encode data **before** applying PCA

Related methods

- Independent component analysis probabilistic independence
- Logistic regression maximizes likelihood of class
- Factor analysis
- Fisher's linear discriminant
- Linear discriminant analysis (LDA)

PCA in Python

```
import numpy as np
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
pca.fit(X)
print(pca.explained_variance_ratio_)
print(pca.singular_values_)
```

Source: <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

Pipeline order

- Impute missing data
- Scale data
- PCA or other dimensionality reduction
- Estimator (fit regression or classification parameters)